

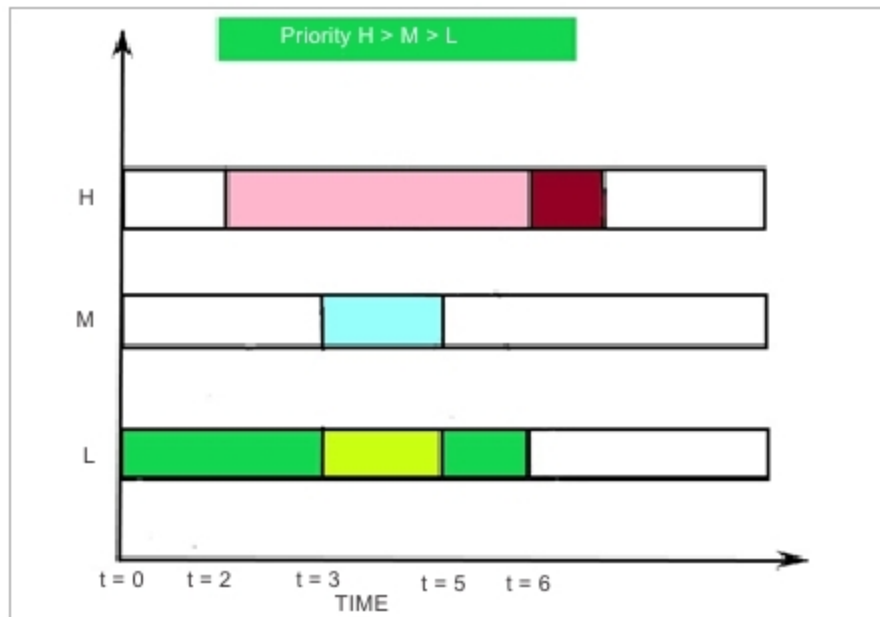


Figure 1: Priority inversion

Figure 1 shows a typical priority inversion scenario. The sequence of events is as follows. The figure depicts three threads: L, M and H. Out of these, L and M share the critical section. L starts executing at $t = 0$ and at $t = 2$, H comes in. Since L holds the lock, H can't pre-empt L and H sleeps. Now at $t = 3$, M comes and pre-empts L since it doesn't share the critical section with L. M finishes at $t = 5$ and L completes its remaining work. Finally, at $t = 6$, H gets to run.

An experimental setup

This entire experiment was carried out on an Ubuntu 17.10 machine with kernel version 4.14.

The system specifications are quad-core Intel (R) Core(TM) i7-4710HQ CPU clocked at 2.50GHz with 8GB of RAM.

The entire source code is available at <https://github.com/sourabhkumar0308/PE> and what follows is a walk-through of the code. There are three files in this source code.

The name of the first file in this source code is *pri-inh-singlecore.c*. In this source code, lines 1 to 7 are the header files. The macros on lines 9 to 11 are the duration of time for which L, M and H will execute respectively.

From lines 13 to 16, policy and mutex variables are declared. *pthread_mutex_t* and *pthread_mutexattr_t* are datatypes used to declare a mutex variable and mutex attribute variable, respectively. *SCHED_FIFO* is a real-time scheduling policy with a priority value in the range 1 (low) to 99 (high). This gives the real-time threads higher priority than the normal threads. With this policy, a thread will continue to

run until it voluntarily yields the CPU or gets pre-empted by a higher priority thread.

The function *stick_thread_to_core* (lines 19 to 30) is used to set the CPU affinity. The function *sysconf* is declared in *unistd.h* and returns the number of cores available, which prevents the user from setting invalid cores. *cpu_set_t* is a data structure that represents a set of CPUs. *CPU_ZERO* clears the set and *CPU_SET* adds the CPU to the set. *pthread_setaffinity_np* sets the CPU affinity of a thread and it takes calling the thread ID as its first argument, the length of the CPU buffer pointed by *CPU_SET* in bytes as the second argument, and address of the data structure as its third argument.

The function *doSomething* (lines 38 to 47) simulates a thread that is busy doing some work. It increases a counter for 'n' seconds. *gettime* is a user defined function that returns the current time in seconds. It prints the sequence in which threads arrive and execute.

Lines 49 to 73 make up a low priority thread. Medium (lines 96 to 112) and high (lines 74 to 95) priority threads also have a similar structure except for a few changes in the case of the former. Medium priority threads don't share a critical section with low and high priority threads. The thing to note here is that all of them are made to run on the same core (0).

sched_param is a structure defined in *sched.h*, and is used to set and get the priority value. Here, two variables are declared first. One is *the_priority*, which is used to set the priority value and the other is *get_prio*, which is used

to get the priority value that is set. *the_priority.sched_priority = sched_get_priority_min(SCHED_FIFO)* sets the priority.

sched_get_priority_min returns the minimum priority value for the *SCHED_FIFO* policy.

pthread_setschedparam applies the priority to the thread and it takes the ID of the calling thread, policy variable and the address of the priority structure as its arguments.

pthread_getschedparam returns the priority of the thread to the *get_prio* structure and it takes the ID of the calling thread, the address of the policy variable and the address of the priority structure as its arguments.

pthread_mutex_lock and *pthread_mutex_unlock* are used to lock and unlock the critical section and they take the *pthread_mutex_t* type variables as their arguments. These functions are defined in *pthread.h*. They take the address of the *pthread_mutex_t* type variable as their argument.

```
sourabh@sourabh-Lenovo-Y50-70: ~
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
```

Figure 2: L is running

```
sourabh@sourabh-Lenovo-Y50-70: ~
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
M is running with priority 89 on cpu: 2
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
L is running with priority 1 on cpu: 0
```

Figure 3: L and M running simultaneously